

 @nate.b.jones

Build the room before you
write the memo. Grab the 4-
prompt project room kit:
source inventory, duplicate
log, missing-context list,
grounded draft.

The first useful agent workflow is not generation. It is
putting the work surface into shape.



NATE

MAY 22, 2026 • PAID

 116 20 4

Share

 Transcript

Nate's Notebook

Welcome to my podcast!
In these audio reviews of
my newsletters, I am to
break down complex AI
topics in a way that's
approachable and
relatable. I want you to
walk away with the
confidence to leverage AI
more effectively at home
and at work!

Listen on



Substack App



Apple Podcasts

When AI produces a mediocre draft from a messy folder,
the prompt is almost never the problem. The room is.

The model has been handed a strategy doc, two slightly different versions of the operating plan, a transcript with two meetings in it, and a deck that no longer matches reality. It is asked to write a memo. To do that, it has to do two jobs at once: figure out what the project is, then produce the artifact. The first job is the hard one. The second job is the one that shows up in the draft.

A sharper prompt won't fix this. You need to prepare the room first.

I recently worked on a project where the real work didn't live in one place. A strategy doc, meeting transcripts, a budget spreadsheet, trip-planning notes, design drafts, old PDFs, follow-up emails, half-finished notes. Some clearly current. Others superseded. A few useful only because they showed how the thinking had changed.

The useful first prompt was much more boring than "write the plan." It was something like: help me build the room. Find the relevant materials. Preserve the originals. Make an inventory. I needed to know which ones were authoritative, which were duplicates, which were old, which were missing. I asked it to summarize each source before synthesizing across them, and I explicitly told it not to write the final deliverable yet.

Only after that did the writing prompt become simple. Give me the current operating plan for the numbers, the transcript for decision context, the older PDF only as



RSS Feed



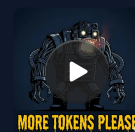
Email mobile setup link

Appears in episode

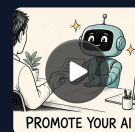


Nate

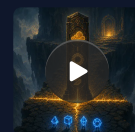
Recent Episodes

**Executive Briefing: You**

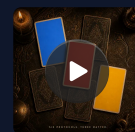
...

**68% of AI power users c**

MAY 21 • NA

**Seven questions...**

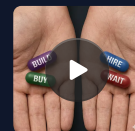
MAY 20 • NA

**Six agent protocols just**

MAY 19 • NA

**What ChatGP sees when it.**

MAY 18 • NA

**Executive Briefing: Stop**

MAY 17 • NA

**Exclusive: a conversation**

MAY 16 • NA

background, and flag unsupported claims rather than smoothing them over. The room made those distinctions visible before the writing started.

This kind of workflow was not really available a year ago. Agents could draft, summarize, and answer questions, but they were uneven at walking a folder tree, opening files in sequence, comparing dates across documents, and inspecting metadata without losing the thread. In the last few months that has changed. The current generation of agents is good at the small, boring, low-level operations the work actually requires. Which means the bottleneck has moved. It is no longer “can the model produce the artifact.” It is “is the source set in shape for the model to do anything useful with it.”

Here’s what’s inside:

What is an AI project room. What a bounded workspace looks like for one serious job, and which tools to use for which source types.

Why AI fails with messy source files. Why serious work fails when you skip the preparation step and jump straight to generation.

How to build an AI source inventory. How to build the artifact that makes everything downstream inspectable.

Summaries, duplicates, and missing context. The three preparation layers that prevent bad synthesis before it starts.

The writing prompt, once the room exists. What changes when you draft from a clean work surface instead of a raw file dump.

Grab the four prompts. A room-builder for file-system tools, an inventory-and-audit for uploaded docs, a grounded-draft prompt that cites every claim back to a source, and a refresh prompt for when new files arrive.

It's build the room.

Subscribers get all posts like these!

LINK: [Grab the Prompts](#)

The hidden cost of skipping preparation is that you never see which source the bad claim came from. The memo reads fine. The number is wrong. Three weeks later you find out the agent pulled from a draft someone perseded in February, and now you are unwinding a decision that was already made on bad data. These four prompts exist because that failure mode is invisible until it's expensive. The room-builder and the inventory-and-audit force the agent to declare authority and surface conflicts before it drafts anything, so when something goes off you can trace it to a file instead of guessing. The grounded-draft prompt cites every claim back to a source ID and flags anything the room does not support, which means review happens against evidence rather than polish. The refresh prompt keeps the room honest

When new files arrive, instead of letting an old working draft quietly outlive the project. Run them in sequence in the messiest folder you have this week. The artifact you want by the end is not a draft. It is a room you can defend.

What is an AI project room

A project room is a bounded workspace for one serious project. It is smaller than a second brain, more specific than a knowledge management system, and not the same problem as building yourself a local AI computer. It is the workspace for one project, set up so that an agent can do useful work inside it.

A project room is the place where the agent gathers the material for one piece of work and makes it legible before anyone asks for a final answer.

For a consulting project, that might mean interview transcripts, client decks, data exports, prior proposals, and meeting notes. For a house purchase: inspection reports, disclosures, contractor estimates, mortgage documents, and email threads. A Substack piece would include all source PDFs, transcripts, draft notes, product docs, screenshots, and prior related posts. A board memo includes the financial model, operating plan, old board deck, current KPI export, and the notes from the last three review meetings.

The point is not to create a perfect archive. The point is to create a usable work surface.

A good project room separates originals from working notes and keeps file provenance intact. It creates an inventory, names the source hierarchy, identifies duplicates, and produces short summaries of each source. It flags missing context and tells you what is ready to use and what still needs human judgment.

Only after that should the agent write.

Tool choice matters. Use [Claude Projects](#) when you need a bounded workspace with uploaded documents and reusable project instructions. Use [ChatGPT Projects](#) or ChatGPT file analysis for smaller source sets and spreadsheets. Use [Cursor](#) or Claude Code when the project room includes code, technical docs, or a folder that needs inspection. Use [NotebookLM](#) when the room is research-heavy and source-bounded, especially when you want to work against many uploaded documents rather than a live file system.

The workflow is overkill for a throwaway note. It is not overkill when the cost of being wrong exceeds the cost of preparing the room: a board memo, hiring packet, investor update, diligence brief, regulatory response, legal review, operating plan, or article that depends on source coverage. Use the room when the final artifact will travel farther than your ability to explain it.

Building an agentic back-office pipeline is a different problem entirely. If you are automating invoice processing, customer support triage, or any operational workflow that runs on a schedule, you need a data strategy that lives upstream of any one project. The team is a unit of work, not a unit of infrastructure. Use the team for the deliverable you are preparing this week. Build the pipeline somewhere else.

Why AI fails with messy source files

Most people still treat AI like a file generator.

They ask it to make a memo, build a spreadsheet, create a deck, draft a proposal, or summarize a meeting.

Sometimes that works. But in serious work, the hard part is often not generating the first draft. The hard part is getting the inputs into a state where generation is worth doing.

This is especially obvious in Office work. The dream version is “AI makes the spreadsheet” or “AI makes the PowerPoint.” The real pain is different. The spreadsheet has messy exports, merged headers, hidden tabs, stale assumptions, formulas that look plausible but are wrong, and numbers that need to be tied back to a source. The deck has brand rules, executive norms, approved language, old charts, new data, and claims that need evidence. The problem is not whether AI can create a

but whether it can handle the material around the
2.

at same pattern applies to almost every knowledge
object.

that you actually need isn't "write." You need a chain of
smaller operations: inspect, gather, normalize, reconcile,
summarize, verify, and only then produce. When you
skip those steps, the AI may still give you something
plausible. It may even look useful. But you have moved the
review burden downstream. Now you have to figure out
whether the answer is grounded, whether the right
sources were used, whether an old file contaminated the
conclusion, and whether the agent missed the one
document that mattered.

You can change the entire interaction by shifting when
the review happens. Review the source inventory
upfront, not after the memo is done. Ask for authority
linking upfront rather than discovering halfway
through that the agent used an obsolete deck. Request a
conflict log and a missing-context list before the
synthesis, not after. These moves prevent the polished
memo from hiding where it came from.

Agents shine when they're working on structure, the
context, and judgment about what's in the room. That's
where they add real value.

How to set up AI file organization

you saw this structure inside Prompt 1. Here's why it's shaped this way.

The first job is to gather without destroying.

That means the agent should not delete files, overwrite originals, or silently collapse duplicates without asking. A useful file-system agent starts conservatively. It looks, copies when appropriate, records paths, and proposes actions before taking irreversible steps.

I usually want a structure like this:

- 00_originals — untouched source files and preserved paths.

- 01_inbox — unsorted material that may or may not matter.

- 02_inventory — the table of sources, authority, relevance, and notes.

- 03_source_summaries — one short summary per important source.

- 04_working_brief — the synthesis layer before drafting.

- 05_outputs — drafts and finished artifacts.

- 99_review — duplicate logs, proposed moves, uncertainty lists, and human approval items.

This looks basic. It should. The folder structure gives the agent a safe place to operate. It also gives you a way to supervise without reading every document first.

How to build an AI source inventory

The most important artifact in this workflow is not the draft. It is the source inventory.

A good source inventory changes the entire interaction. It tells you what the agent thinks the project consists of. That gives you a chance to correct the working set before the final artifact inherits the mistake.

At minimum, the inventory should include file path, source type, date, owner if known, relevance, authority level, current or superseded status, supported claims, citations, intended use in the final work, and notes for human review.

A final signed agreement outranks a negotiation draft. The current spreadsheet beats a screenshot of last month's numbers. A transcript may be more faithful to what was said, while a cleaned meeting note may be more useful for decisions and owners. The approved check represents the story even if the underlying data lies somewhere else, and an old PDF may be useful background but not a source for current claims.

AI can help sort this out, but it should not be allowed to make the sorting. The inventory makes the judgment defensible.

The trust habit is to review the inventory before synthesis. Spot-check the files the agent marks as

authoritative. Open one source it calls current and one source it calls superseded. Check whether dates, owners, and titles match what you know. If the agent cannot explain why one file outranks another, do not let it draft from the room yet.

For a writing project, this is especially important. A model can synthesize across twenty sources in a way that feels smooth but erases the difference between confirmed facts, reported claims, working assumptions, and your own interpretation. The inventory gives you a chance to say, “Use this file for the timeline, this one for the numbers, this transcript for the quote, and ignore that older draft except as background.”

Much better than trying to debug the final prose.

Summaries before synthesis

The next step is source summaries.

This sounds like ordinary AI summarization, but the purpose is different. You are not asking for a general summary because you are too busy to read. You are asking the agent to prepare the source for later use.

A useful project-room summary should answer five questions.

What is this source?

What does it contain that matters for the project?

What claims, numbers, or decisions does it support?

What are its limitations?

How should it be used in the final work?

The last question is important. A transcript gives you the name and raw detail but not polished phrasing. A spreadsheet carries the current figures but not the strategic interpretation behind them. A slide deck presents the approved frame without the underlying evidence. A screenshot proves that something was visible at a moment in time, but not that it remains true.

The agent should also pull out uncertainty. A garbled name in a transcript should be flagged, not guessed at. A document that appears to be a draft should be labeled as one. Disagreements between documents should stay visible, not get smoothed over. Missing sources should be called out.

The discipline here is simple: do not let the agent become confident before the room is clean.

1 duplicate file detection for projects

Most people think duplicate detection is housekeeping. In AI work, duplicates are a reasoning problem.

If the agent sees three versions of a plan and does not know which one is current, it may blend them. The same

inscript exported twice with different names gets overweighted. An old deck and a new deck with similar titles become a source for wrong claims. A revised budget spreadsheet next to an earlier copy can produce averaged assumptions without any flag.

And synthesis starts here.

The fix is not to let the agent delete duplicates. The fix is to make it report them.

A good duplicate log should separate exact duplicates, likely duplicates, and version families. Exact duplicates appear identical. Likely duplicates share similar names, titles, or content. Version families represent the same artifact over time.

The agent should propose a current version, but it should explain why. Newer modified date is not always enough. The current version might have “final” in the title, or it might be the version attached to a later email, or the one whose contents match the latest meeting minutes. Sometimes it is not possible to know without human review.

This is exactly the kind of work AI is good at. It can scan names, contents, metadata, and surrounding context quickly. But the final authority decision often belongs to the human.

That division of labor is healthy. Let the agent find the mess. Do not let it silently resolve the mess when the

Consequences matter.

The missing-context list

One of the best signs that an AI agent is helping properly is that it tells you what it does not have.

A weak workflow produces an answer from whatever happens to be in front of it. A strong workflow produces a missing-context list before the answer.

For a serious project, the agent walks through the room and notices three kinds of problems. First, what is missing: the decision no one documented, the number with no source, the absent owner. Second, what is ambiguous: which version is current, which draft is outdated, where documents disagree. Third, what is dangerous: unsupported claims, private sources that could not be used, inferences presented as facts.

That matters because the missing material is often more important than the available material. Your document may say “as discussed” while the source of truth sits in the discussion itself. A transcript may say “we agreed to this,” but the actual decision may have changed later in the week. The spreadsheet includes a number without the assumptions. The deck includes a chart without the data source.

If you ask for the final memo too early, these gaps become hallucination traps. If you ask for the missing-context list first, they become review items.

The writing prompt, once the room exists

Once the project room exists, the writing prompt gets shorter and better.

Instead of saying:

“Write a strategy memo from these files.”

You say:

“Use the reviewed source inventory and working brief. Treat the current operating plan as authoritative for members, the transcript as source material for decision context, and the older deck as background only. Draft the memo, cite claims back to source IDs, and flag anything not supported by the room.”

The model is no longer guessing what the source set means. You have already reviewed the room. You have already decided which materials matter. The agent has already summarized the evidence, identified conflicts, and listed gaps. Now the writing work can be grounded.

This is also better for review. When the draft says something questionable, you can trace it back to the inventory or the source summary. Inferences should be labeled. Unsupported claims should be flagged. When the source hierarchy changes, you can update the project room and regenerate from a cleaner base.

st-moving projects need maintenance. If new files arrive daily, refresh the inventory before each serious drafting pass. If the source hierarchy changes, update the working brief. A project that changes purpose needs a new room rather than a contaminated old one. The goal is not a permanent archive but a clean enough work surface for the next decision.

You are not trying to make AI perfect. You are trying to make its work inspectable.

Pick one project this week

Block 45 minutes Tuesday afternoon. Pick the project where you have the messiest folder and the most insequential deliverable. Drop the relevant materials into Claude Projects, ChatGPT Projects, Cursor, or NotebookLM depending on the source set. Use the prompts in the kit above. Do not let the agent synthesize until you have reviewed the inventory.

That single session will teach you more about how your own work is organized than another hour of prompt tweaking. The artifact you want by the end is not a draft. It is a source inventory, duplicate log, missing-context list, and working brief.

The learning ramp most people need is not “how do I make the AI sound smarter?” but “how do I make the work surface clean enough that AI can help?”

The shift from generation to preparation

The old AI question was whether the model could produce the artifact.

Can it write the memo? Make the spreadsheet? Create the deck? Summarize the transcript?

Those questions still matter, but they are no longer the most interesting ones. The more useful question is whether the agent can help prepare the conditions under which good work happens.

Can it find the right sources, tell which ones are current, preserve originals, and identify missing context instead of inventing around it?

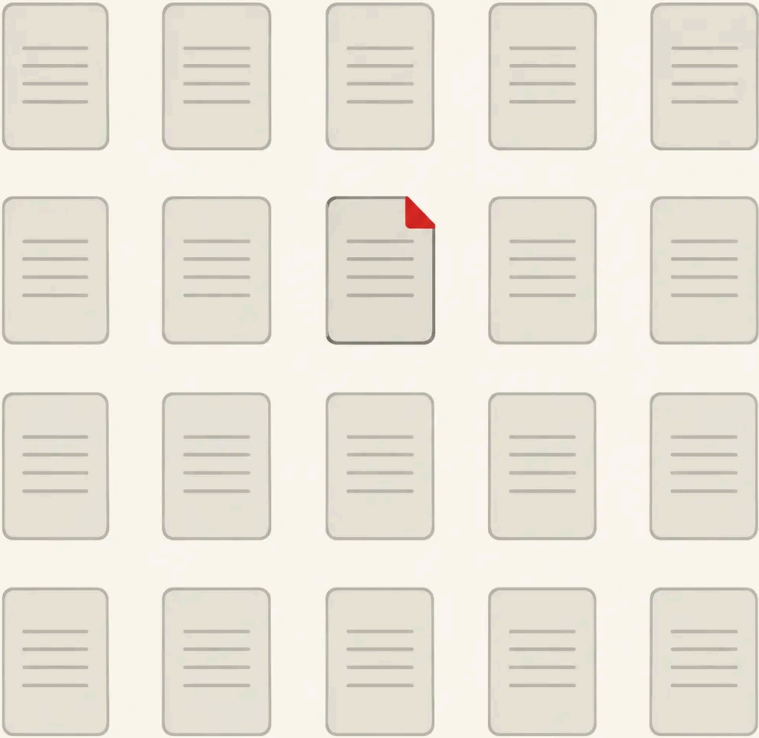
I think many people will first feel agents become useful right there. Not as magical writers or autonomous employees. Not as another demo where a model produces a passable first draft from a clean prompt.

It's the assistant that walks into the messy room before a meeting starts, turns on the lights, labels the boxes, finds the missing folder, puts the current documents on the table, and tells you what still does not add up.

Then you can write.

Not before.

I make this Substack thanks to readers like you!
[Learn about all my Substack tiers here](#)



116 Likes · 4 Restacks

Discussion about this video

Comments Restacks

Write a comment...

Courtney Caldwell  Eventual Eureka 2d ...

Man, this is one that really HITS HARD. What a common-sensical way to avoid the garbage in/garbage out (and all the invisible assumptions).

 LIKE (7)  REPLY

 SHARE

mikael thomas  2d Edited ...

That's very interesting, and it's exactly why I did not move further into production layers until I realized that you need a complete operational ontology, where the meaning and relationships between every element are declared within a structured context. From there, the system can grow systematically, becoming a magnet for discipline — stronger than any harness, though not necessarily replacing one.

If every angle of the system continuously maps back into this structured semantic framework, the model begins to perceive it as the most rewarding path, because it gains access to meaning, methods, skills, and intent as part of an integrated matrix of tools and relationships. At that point, it is no longer primarily about prompting, but about managing the model's attention through a declared symbolic structure.

This effectively channels the model's raw statistical inference into the domain context. That is where the alpha lives.

 LIKE (3)  REPLY

 SHARE

Kelly Heaton & ChatGPT-4o  The Coherence Code 2d ...

This is why machine learning often fails to yield insights. The model cannot make meaning from raw data without an intelligent organizational structure.

 LIKE (3)  REPLY

 SHARE

Sr  2d ...

Loved this episode and it made me feel validated as I have been going in this direction with a similar approach I just blogged about <https://stonebergdesign.com/blog/project-kickoff-skill> I will be adopting the "data room" naming scheme

 LIKE (2)  REPLY (1)

 SHARE



Aaron Lawson Fork in the Road 1d ...

What are the function differences you see in your own project kickoff vs what Nate proposed?



LIKE



REPLY (1)



SHARE



Sr 1d ...

Nate talked about a file manifest or inventory with definitions as well as duplicate issues and I have not addressed those explicitly. They do get captured as a side benefit of the activity log though and that has been enough for me so far.



LIKE (1)



REPLY



SHARE



Thorsten 2d ...

This is timely insight as I'm attempting to build a research assistant with Claude inside of an Obsidian vault, using Zotero (and it's integration with Obsidian) to manage sources and citations. I'll attempt to start with Nate's proposed structure for each writing "project" and might at a later stage also try to work a Discourse Graph into the workflow.



LIKE (1)



REPLY



SHARE



Michael Jones Michael Jones 15h ...

SMH you did it again another tool for our toolbox Class in session



LIKE



REPLY



SHARE



Aaron Lawson Fork in the Road 1d ...

As a bumbling vide coder moving at 90mph, I'm interested in hearing whether this schema makes sense for a new coding project repo, or if there are additional protocols that would make sense to swap out or add?



LIKE



REPLY



SHARE



The Geek in Review The Geek in Review 1d ...

I have been working on a project and applying (or so I thought) a lot of these principles in my workflow.

I had Claude 4.7 stop the project and review Nate's post and prompts. After one audit run, here's the first thing it said back "The audit produced findings I would have completely missed by proceeding to Phase E. Nate's framework just paid for itself."

♡ LIKE 💬 REPLY

↑ SHARE

mikael thomas  1d Edited ...

The useful first prompt was much more boring than "write the plan." It was something like: help me build the room. Find the relevant materials. Preserve the originals. Make an inventory. I needed to know which files were authoritative, which were duplicates, which were old, which were missing. I asked it to summarize each source before synthesizing across them, and explicitly told it not to write the final deliverable yet.

On this, i have completely generalised this process in three parts, the domain context that govern the agent behaviour and normalise all the static types elements, attached a redis memory for the current context in two layers, system and agents level, they can navigate my knowledge base and store what they need for the current job or consolidate my intent from reading transcripts of the past sessions, and the last that I just added that covers above is a store of object centric matrix. Like in this case observation matrix where I force the model to work on its one perception of the object, analyse dimensions, etc and I keep updating hit through until the project is done, then I store it as a type, then evaluate matrix or maintenance matrix etc.

This layer is portable cross agents and recoverable in time and extremely helpful to manage the model attention on one goal, especially when it's abstract.

♡ LIKE 💬 REPLY

↑ SHARE

Robert LaSalle  2d ...

I just watched Nate's post today and the artifacts he suggests creating (like the file inventory) are gonna come in very handy when he discusses /goal in Claude Code...

<https://code.claude.com/docs/en/goal>

♡ LIKE 💬 REPLY

📌 SHARE

 Pamela Pierce 2d

I just spent a couple weeks creating the structure. Finally it's going to the AI for sorting.

♡ LIKE 💬 REPLY

📌 SHARE

 Alexander Verharen 2d

I've been building a solution that takes those original documents, working briefs and final docs to allow for truthful reporting. I built an entire digest-driven infrastructure for that. Every project has it's own pitfalls and my digest service provides preambles for prompts depending on your objective with these materials. It's very insightful to learn how I can get even more value out of this. I'd like to be able to do this with Notebook LM and tools like that

♡ LIKE 💬 REPLY

📌 SHARE

 Petko Mikhailov 2d

How can one make this work with Copilot Cowork in a Microsoft ecosystem (OneDrive, SharePoint, Teams, Loop, Power Automate)?

♡ LIKE 💬 REPLY

📌 SHARE

 Trtypn 2d

I just built this as a 'Workstation' based on Jeff Yu's "Claude Cowork System". I will play with it this weekend on a side project I'm spinning up! @Nate - you left Cowork out of the equation, but I think Jeff's framework + your 'Clean Room' approach make a great combo!

Here's Jeff's video for reference

https://youtu.be/0_dSWLOHKng?si=t6ofgiAGvJaiPOxa

♡ LIKE 💬 REPLY

📌 SHARE

 Al Simon AI's Substack 2d

thanks nate, this is directly relevant to what i've been building -
i had 5 parts of this already and your article gave me another 4.
just about to finish building it

♡ LIKE 💬 REPLY

🔗 SHARE

 **Saul Lowery**  Saul Lowery 2d ...

This is literally how I started building my Trust Fund Baby
Architecture. All the way back to the first factory, you have to
have a clean factory floor with everything in place, so the agent
can do its job. Anything outside of that and you're setting it up
for failure.

♡ LIKE 💬 REPLY

🔗 SHARE

 **John Halbert**  2d *Edited* ...

funny, i turned to files as a means to make working with llms
more reliable and effective early on, and here we are with files
being first class in most harnesses in one way or another.

♡ LIKE 💬 REPLY

🔗 SHARE

 **Dina**  2d ...

NOT BORING!

♡ LIKE 💬 REPLY

🔗 SHARE

© 2026 Nate · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture